

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Auditor Scan

#	Scan ID		Date
	6		June 30, 2026
	Organization		Repository
	CMTA		CMTAT-Factory
	Branch		Commit Hash
	main		18a8e66c...07016ec8

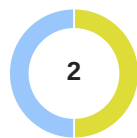
Contracts in scope

contracts/libraries/CMTATBeaconFactoryBase.sol contracts/libraries/CMTATFactoryBase.sol
contracts/libraries/CMTATFactoryInvariant.sol contracts/libraries/CMTATFactoryRoot.sol
contracts/libraries/CMTATTransparentFactoryBase.sol contracts/libraries/ContractVersion.sol
contracts/libraries/FactoryErrors.sol contracts/light/CMTAT_LIGHT_BEACON_FACTORY.sol
contracts/light/CMTAT_LIGHT_TP_FACTORY.sol contracts/standard/CMTAT_BEACON_FACTORY.sol
contracts/standard/CMTAT_TP_FACTORY.sol contracts/standard/CMTAT_UUPS_FACTORY.sol

Code Statistics

Findings	Contracts Scanned	Lines of Code
2	12	868

Findings Summary



Total Findings

- High Risk (0)
- Medium Risk (0)
- Low Risk (1)
- Info (1)
- Best Practices (0)

Code Summary

The protocol provides a specialized factory infrastructure designed to deploy and manage Capital Markets and Technology Association Token (CMTAT) instances. It offers a robust framework for launching upgradeable token contracts using multiple industry-standard proxy patterns. By accommodating both Standard and Light variations of the CMTAT, the architecture provides developers with flexibility depending on their specific tokenization needs.

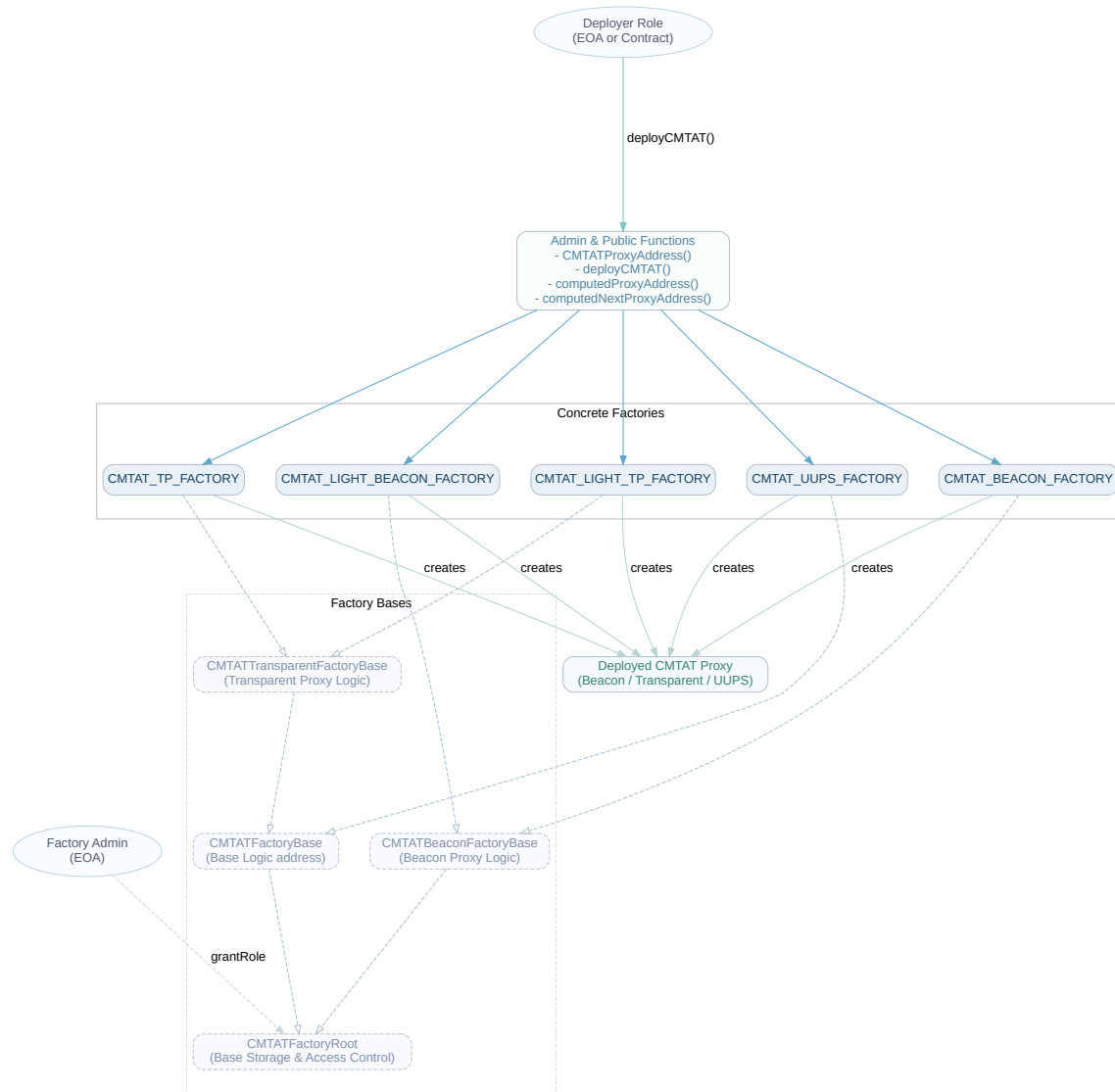
The core architecture relies on an inheritance tree originating from a root factory that handles access control, deployment indexing, and deterministic address calculation using CREATE2.

Depending on the desired upgradeability mechanism, the protocol separates its deployment logic into Beacon, Transparent Proxy (TP), and Universal Upgradeable Proxy Standard (UUPS) factories. The system maintains an on-chain registry of all deployed tokens through an internal counter and handles both custom-provided salts and auto-generated sequential salts to guarantee address collision avoidance and determinism across multiple networks.

Main Entry Points and Actors:

- `deployCMTAT` in `CMTAT_LIGHT_BEACON_FACTORY`: Allows actors with the deployer role to deploy a Light version of the token utilizing a Beacon proxy for synchronized upgrades across multiple instances.
- `deployCMTAT` in `CMTAT_LIGHT_TP_FACTORY`: Allows actors with the deployer role to instantiate a Light version of the token using a Transparent Upgradeable Proxy for isolated upgradeability.
- `deployCMTAT` in `CMTAT_BEACON_FACTORY`: Enables actors with the deployer role to deploy the Standard token implementation behind a Beacon proxy.
- `deployCMTAT` in `CMTAT_TP_FACTORY`: Enables actors with the deployer role to create a Standard token deployment utilizing a Transparent Upgradeable Proxy along with a new ProxyAdmin.
- `deployCMTAT` in `CMTAT_UUPS_FACTORY`: Permits actors with the deployer role to deploy the Standard token version using the UUPS proxy pattern for cost-effective deployment and upgrade management.

Code Diagram





1 of 2

Findings

contracts/libraries/CMTATFactoryRoot.sol contracts/standard/CMTAT_UUPS_FACTORY.sol

contracts/standard/CMTAT_TP_FACTORY.sol

contracts/standard/CMTAT_BEACON_FACTORY.sol

contracts/light/CMTAT_LIGHT_TP_FACTORY.sol

contracts/light/CMTAT_LIGHT_BEACON_FACTORY.sol

Counter-derived next-address helpers are front-runnable across authorized deployers in CMTATFactoryRoot

- Low Risk

Root cause: when `useCustomSalt` is `false`, the factory does not reserve a predicted address for a specific caller. Both `computedNextProxyAddress(...)`-style helpers and the eventual `deployCMTAT(...)` call derive the effective CREATE2 salt only from the shared global `cmtatCounterId`, and the caller-supplied `deploymentSaltInput` is ignored in that mode. As a result, every concrete factory that exposes `computedNextProxyAddress(...)` inherits a race where another authorized deployer can consume the current counter-derived salt first and shift the victim onto a different deployment address.

```
function nextDeploymentSalt() public view virtual returns(bytes32 saltBytes) {
    return keccak256(abi.encodePacked(cmtatCounterId));
}

function _checkAndDetermineDeploymentSalt(bytes32 deploymentSalt) internal virtual
returns(bytes32 saltBytes){
    if(useCustomSalt){
        if(customSaltUsed[deploymentSalt]){
            revert FactoryErrors.CMTAT_Factory_SaltAlreadyUsed();
        }else {
            customSaltUsed[deploymentSalt] = true;
            saltBytes = deploymentSalt;
        }
    }else{
        saltBytes = nextDeploymentSalt();
    }
}

function _computeDeploymentSalt(bytes32 deploymentSaltInput) internal view virtual
returns(bytes32 saltBytes){
    return useCustomSalt ? deploymentSaltInput : nextDeploymentSalt();
}
```

Exploit path: (1) Alice and Bob both hold `CMTAT_DEPLOYER_ROLE` on a factory configured with `useCustomSalt == false`. (2) Alice calls `computedNextProxyAddress(...)` off-chain and pre-whitelists, pre-funds, or otherwise relies on the predicted address `A`. (3) Bob front-runs with any successful `deployCMTAT(...)` call on the same factory, consuming the current `cmtatCounterId`-derived salt. (4) Alice's later `deployCMTAT(...)` transaction still succeeds, but it now uses the next counter-derived salt and deploys at `B != A`. (5) Because the factory never reuses the consumed counter value, assets or permissions prepared for `A` can remain stranded at an address the factory will never deploy to.

Impact: any non-custom-salt factory exposes a role-holder griefing/front-running vector against deterministic predeployment workflows. This does not corrupt factory state, but it breaks the documented 'predict next address' guarantee in the multi-deployer model and can cause denial of service or permanent loss of tokens/permissions sent to a stale predicted address.

Severity Note:

- Integrators or users pre-act (e.g., pre-fund ETH/tokens or pre-whitelist) based on `computedNextProxyAddress` before the deployment is mined.
- There are multiple holders of `CMTAT_DEPLOYER_ROLE`, and at least one behaves adversarially or negligently.
- Admin does not immediately revoke the malicious deployer before the race.

contracts/libraries/CMTATFactoryRoot.sol contracts/standard/CMTAT_UUPS_FACTORY.sol
contracts/standard/CMTAT_TP_FACTORY.sol
2 of 2
✦ Findings contracts/standard/CMTAT_BEACON_FACTORY.sol
contracts/light/CMTAT_LIGHT_TP_FACTORY.sol
contracts/light/CMTAT_LIGHT_BEACON_FACTORY.sol

Counter-derived salts can be reused during reentrant proxy initialization

[Info](#)

When custom salts are disabled, the factory derives the CREATE2 salt from the current `cmtatCounterId`, but the counter is not incremented until after the proxy deployment and initializer execution have completed. Proxy creation executes external code: `ERC1967Proxy`, `TransparentUpgradeableProxy`, and `BeaconProxy` constructors all execute initializer calldata against the implementation during construction. If that initialization path can reenter a factory deployment function through an address that holds `CMTAT_DEPLOYER_ROLE`, the reentrant deployment observes the same `cmtatCounterId` as the outer deployment and therefore derives the same automatic salt.

Relevant code path:

```
function _checkAndDetermineDeploymentSalt(bytes32 deploymentSalt) internal virtual
returns(bytes32 saltBytes){
    if(useCustomSalt){
        if(customSaltUsed[deploymentSalt]){
            revert FactoryErrors.CMTAT_Factory_SaltAlreadyUsed();
        }else {
            customSaltUsed[deploymentSalt] = true;
            saltBytes = deploymentSalt;
        }
    }else{
        saltBytes = nextDeploymentSalt();
    }
}

function nextDeploymentSalt() public view virtual returns(bytes32 saltBytes) {
    return keccak256(abi.encodePacked(cmtatCounterId));
}

function _deployAndRegisterProxy(bytes memory bytecode, bytes32 deploymentSalt) internal
returns (address cmtatAddress) {
    cmtatAddress = Create2.deploy(0, deploymentSalt, bytecode);
    emit CMTATDeployed(cmtatAddress, msg.sender, cmtatCounterId, deploymentSalt);
    ++cmtatCounterId;
    cmtatsList.push(cmtatAddress);
}
```

Because `Create2.deploy` happens before `++cmtatCounterId`, reentrant deployments in automatic-salt mode can successfully use the same effective salt as the in-flight outer deployment as long as their proxy init code differs, producing distinct CREATE2 addresses. This breaks the protocol invariant that consecutive successful automatic deployments use different counter-derived salts. It can also make event logs contain the same salt for multiple deployment IDs and can skip the salt that would otherwise correspond to the next counter value,

disrupting off-chain workflows that rely on the documented one-counter-one-salt relationship for deterministic deployments.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.